INSTITUTO FEDERAL
Minas Gerais
Campus São João Evangelista

Valdir de Souza Carvalho Neto **SI 241 - Algoritmos e Estruturas de Dados II**

Revisão Prova 02

Bubblesort

O Bubblesort é o método de ordenação mais simples de entender, mais fácil de implementar, e talvez o mais conhecido

Princípio Básico

O princípio básico deste método consiste na troca de valores entre posições consecutivas. Isso faz com que os valores mais altos (ou mais baixos) "borbulhem" para o fim do conjunto.

Funcionamento (Passo a Passo)

1. Percorra o vetor inteiro comparando elementos adjacentes (dois a dois).
2. Troque as posições dos elementos se eles estiverem fora de ordem.
3. Repita os dois passos acima com os primeiros n−1 itens, depois com os primeiros n−2 itens, e assim por diante, até que reste apenas um item.

Exemplo de Ordenação (Passo a Passo)

Vamos usar o Exemplo 1 fornecido no material com o vetor inicial: 10,8,9,2,4,1.

Vetor Inicial: 10,8,9,2,4,1

Primeira Passada (comparando até o final):

- 10,8: Troca →8,10,9,2,4,1
- 10,9: Troca →8,9,10,2,4,1
- 10,2: Troca →8,9,2,10,4,1
- 10,4: Troca →8,9,2,4,10,1
- 10,1: Troca →8,9,2,4,1,10
- O 10 "borbulhou" para a última posição.

Segunda Passada (comparando até o penúltimo):

- 8,9: Não troca →8,9,2,4,1,10
- 9,2: Troca →8,2,9,4,1,10
- 9,4: Troca →8,2,4,9,1,10
- 9,1: Troca →8,2,4,1,9,10
- O 9 "borbulhou" para sua posição correta (considerando o final).

Terceira Passada (comparando até o antepenúltimo):

- 8,2: Troca →2,8,4,1,9,10
- 8,4: Troca →2,4,8,1,9,10

- 8,1: Troca →2,4,1,8,9,10

Quarta Passada (comparando até a quarta posição):

- 2,4: Não troca →2,4,1,8,9,10
- 4,1: Troca →2,1,4,8,9,10

Quinta Passada (comparando até a terceira posição):

- 2,1: Troca →1,2,4,8,9,10

Vetor Final Ordenado: 1,2,4,8,9,10.

Análise de Complexidade

A análise do Bubblesort é simplificada pela presença de dois laços for com limites inferior e superior.

- O laço mais externo executa n−2 vezes.
- Para cada i entre n e 2, o laço mais interno interage j = i − 1 vezes.

Como as comparações de chaves são feitas no laço mais interno, o número total de comparações é dado por:

$\sum i = (i-1) = (n-1) + (n-2) + \cdots + 2 + 1 = (n(n-2)) / 2 = O(n^2)$.

Portanto, a complexidade de comparação é $O(n^2)$.

Movimentação:

- **Melhor caso:** O(0) (se o vetor já estiver ordenado, não há trocas).
- **Caso médio:** $O(n^2)$.
- **Pior caso:** $O(n^2)$.

Considerações (Vantagens e Desvantagens)

- É um método muito simples.
- Adequado para arquivos pequenos.
- Tem um custo alto em termos de tempo de execução.
- O número de operações de comparações não se altera se o vetor já estiver parcialmente ordenado.

Select

A Seleção Direta é um processo de classificação que consiste em uma seleção sucessiva do menor (ou do maior) valor contido no vetor, dependendo se a classificação dos elementos será em ordem crescente ou decrescente.

Algoritmo de Seleção Direta (Passo a Passo)

1. É feita uma varredura no segmento que contém elementos ainda não selecionados, identificando o elemento de menor (ou maior) valor.
2. É realizada a troca do elemento identificado na etapa anterior com o primeiro elemento do segmento.
3. O tamanho do segmento que contém os elementos ainda não selecionados é atualizado, ou seja, subtrai-se um de seu tamanho.
4. O processo é repetido até que o segmento fique com apenas um elemento, que é o maior (ou menor) valor do vetor.

Exemplo de Ordenação (Passo a Passo)

Vamos usar o Exemplo 1 fornecido no material com o vetor inicial: 34,8,64,51,32,21.

Vetor Original: 34,8,64,51,32,21

Depois i = 0:

- O menor elemento no segmento (todo o vetor) é 8.
- 8 é trocado com o primeiro elemento do segmento (34).
- Vetor: 8,34,64,51,32,21.
- O segmento "não selecionado" agora começa a partir do 34.

Depois i = 1:

- O menor elemento no segmento restante (34,64,51,32,21) é 21.
- 21 é trocado com o primeiro elemento desse segmento (34).
- Vetor: 8,21,64,51,32,34.
- O segmento "não selecionado" agora começa a partir do 64.

Depois i = 2:

- O menor elemento no segmento restante (64,51,32,34) é 32.
- 32 é trocado com o primeiro elemento desse segmento (64).
- Vetor: 8,21,32,51,64,34.
- O segmento "não selecionado" agora começa a partir do 51.

Depois i = 3:

- O menor elemento no segmento restante (51,64,34) é 34.
- 34 é trocado com o primeiro elemento desse segmento (51).
- Vetor: 8,21,32,34,64,51.
- O segmento "não selecionado" agora começa a partir do 64.

Depois i = 4:

- O menor elemento no segmento restante (64,51) é 51.
- 51 é trocado com o primeiro elemento desse segmento (64).
- Vetor: 8,21,32,34,51,64.
- O segmento "não selecionado" agora tem apenas um elemento, e o processo termina.

Vetor Final Ordenado: 8,21,32,34,51,64.

Análise de Complexidade

A análise é simplificada pela presença de dois laços for com limites inferior e superior.

- O laço mais externo executa n−1 vezes.
- Para cada i entre 1 e (n−1), o laço mais interno interage j=n−i vezes.

Como as comparações de chaves são feitas no laço mais interno, o número total de comparações é dado por:

$\sum i = (n-i) = (n-1) + (n-2) + \cdots + 2 + 1 = (n(n-1) / 2 = O(n^2)$.

Portanto, a complexidade de comparação é C(n)=O(n2).

Em relação às movimentações, há 3 movimentos dentro do laço externo, e como ele vai de i = 1 até n−1, o número total de movimentos é M(n) = 3(n−1).

Considerações (Vantagens e Desvantagens)

- É adequado para ser utilizado em arquivos com registros grandes.
- Possui um custo linear no tamanho da entrada para o número de movimentos de registros.
- O número de movimentos dos registros é pequeno.
- Para aplicações onde o custo de movimentar supera o custo de comparações, nenhum outro algoritmo ordena um arquivo com tão poucos movimentos quanto este.
- O fato de o arquivo estar ordenado não diminui o número de movimentos.
- O algoritmo **não é estável**. Um algoritmo é estável se preserva a ordem de registros de chaves iguais. Isto é, se tais registros aparecem na sequência ordenada na mesma ordem em que estavam na sequência inicial. Por exemplo, uma sequência 3[a],2[b],2[c],1[d] ordenado por um algoritmo estável resultaria em 1[d],2[b],2[c],3[a], enquanto um instável poderia resultar em 1[d],2[c],2[b],3[a].

Insert Sort

O Insert Sort, ou Inserção Direta, é um método de ordenação que se baseia na inserção de cada elemento em sua posição correta, considerando os elementos que já estão ordenados.

Definição

Este método consiste em realizar a ordenação inserindo cada um dos elementos em sua posição correta, levando em consideração os elementos já ordenados.

Etapas da Ordenação (Funcionamento)

1. Neste algoritmo, a partir de i=2, o i-ésimo item da sequência de origem é pego e transferido para a sequência de destino, sendo inserido em seu lugar apropriado.
2. A colocação do item em seu lugar apropriado na sequência de destino é feita movendo-se os itens maiores para a direita e, em seguida, inserindo o item na posição que ficou vazia.

3. Nesse processo de alternar comparações e movimentos de registros, há duas condições distintas que podem causar a terminação do processo:
    - Um item menor que o item em consideração é encontrado.
    - O final da sequência de destino é atingido à esquerda.
4. Uma boa solução para a situação de um laço com duas condições de terminação é usar um registro sentinela: na posição zero do vetor, coloca-se o próprio registro em consideração.

Exemplo de Ordenação (Passo a Passo)

Vamos acompanhar o exemplo fornecido com o vetor inicial: 34,8,64,51,32,21.

Vetor Original: 34,8,64,51,32,21

Depois de p = 1:

- O elemento 8 é pego e comparado com 34. Como 8<34, 34 é movido para a direita e 8 é inserido na primeira posição.
- Vetor: 8,34,64,51,32,21.
- Posições Movidas: 1.

Depois de p = 2:

- O elemento 64 é pego. Ele é maior que 34, então não há movimentos e 64 permanece em sua posição relativa aos elementos já ordenados (8,34).
- Vetor: 8,34,64,51,32,21.
- Posições Movidas: 0.

Depois de p = 3:

- O elemento 51 é pego. É menor que 64, então 64 é movido para a direita. 51 é comparado com 34, é maior, então 51 é inserido após 34.
- Vetor: 8,34,51,64,32,21.
- Posições Movidas: 1.

Depois de p = 4:

- O elemento 32 é pego.
- Comparado com 64→ move 64 para a direita.
- Comparado com 51→ move 51 para a direita.
- Comparado com 34→ move 34 para a direita.
- Comparado com 8→ é maior, então 32 é inserido após 8.
- Vetor: 8,32,34,51,64,21.
- Posições Movidas: 3.

Depois de p = 5:

- O elemento 21 é pego.
- Comparado com 64→ move 64 para a direita.
- Comparado com 51→ move 51 para a direita.

- Comparado com 34→ move 34 para a direita.
- Comparado com 32→ move 32 para a direita.
- Comparado com 8→ é maior, então 21 é inserido após 8.
- Vetor: 8,21,32,34,51,64.
- Posições Movidas: 4.

Vetor Final Ordenado: 8,21,32,34,51,64.

Análise de Complexidade

A complexidade do Insert Sort é analisada em termos de comparações e movimentações.

Análise de Comparação:

- **Melhor caso:** $C(n) = n - 1$. Isso ocorre quando o arquivo já está ordenado. O item em consideração é comparado apenas com o último elemento da sequência destino e não há movimentos.
- **Pior caso:** $C(n) = n^2/2 + n/2 - 1$. Ocorre quando o arquivo está em ordem inversa, e cada item precisa ser movido até o início da sequência ordenada.
- **Caso médio:** $C(n) = n^2/4 + 3n/4 - 1$.

Simplificando com a notação Big O:

- **Melhor caso:** $O(n)$.
- **Pior caso:** $O(n^2)$.
- **Caso médio:** $O(n^2)$.

Análise de Movimentação:

- **Melhor caso:** $M(n) = 3(n-1)$.
- **Pior caso:** $M(n) = n^2/2 + 5n/2 - 3$.
- **Caso médio:** $M(n) = n^2/4 + 11n/4 - 3$.

Conclusões (Vantagens e Desvantagens)

- É o método a ser utilizado quando o arquivo está "quase" ordenado.
- É um bom método quando se deseja adicionar alguns itens a um arquivo já ordenado, pois o custo é linear nesse cenário.
- O algoritmo de ordenação por inserção é **estável**. Isso significa que ele preserva a ordem de registros com chaves iguais.
- São feitos muitos deslocamentos cada vez que um elemento é colocado na sua posição correta, o que pode resultar em um número muito significativo de movimentos.

ShellSort

O Shellsort foi criado por Donald Shell em 1959 e publicado pela Universidade de Cincinnati. Ele é considerado o algoritmo de classificação mais eficiente entre os de complexidade quadrática.

Princípio Básico

Basicamente, o algoritmo percorre a lista várias vezes, dividindo o grupo maior em grupos menores. Nesses grupos menores, é aplicado o método de ordenação por inserção.

O Shellsort é uma extensão do algoritmo de ordenação por inserção. O problema com a ordenação por inserção é que ela troca itens adjacentes para determinar o ponto de inserção. Isso pode levar a n−1 comparações e movimentações quando o menor item está na posição mais à direita no vetor. O Shellsort contorna esse problema permitindo trocas de registros distantes um do outro.

Ou seja, a principal diferença em relação à inserção direta é o número de segmentos do vetor. Enquanto na inserção direta se considera um único segmento onde os elementos são inseridos ordenadamente, no Shellsort são considerados diversos segmentos.

Funcionamento (Etapas da Ordenação)

1. Primeiro, compara elementos separados por "h" posições e os rearranja.
2. Em seguida, a distância de comparação (h) é progressivamente diminuída até que h=1, o que corresponde ao algoritmo de inserção direta.
3. A ordenação é efetuada dividindo o conjunto em subconjuntos.
4. Os subconjuntos são constituídos por elementos separados por "n" elementos (incremento).
5. Esses subconjuntos são ordenados utilizando o método de inserção.
6. Os passos 1, 2 e 3 são repetidos, com a diminuição do incremento, até que o conjunto esteja completamente ordenado.

Como Obter o "H" (Sequência de Incrementos)

Ninguém descobriu a sequência perfeita para utilização neste caso, mas várias foram estudadas. Os melhores resultados foram obtidos usando a **sequência de Knuth**: (1,4,13,40,121,364,1093).

- Comece com h = 1.
- Obtenha uma sequência de números usando a fórmula h = 3∗h + 1.
- Pare quando h for maior que o tamanho do array e use o último número obtido.
- Para obter a sequência em ordem decrescente para aplicar ao algoritmo, use a fórmula inversa: h = (h − 1) / 3.

Exemplo de Ordenação (Passo a Passo)

Vamos usar um exemplo com as chaves iniciais: O,R,D,E,N,A.

Vetor inicial: O R D E N A (tamanho N = 6)

Sequência de Knuth para N = 6:

Começamos com h = 1.

h = 3∗1 + 1 = 4.

O próximo h=3∗4+1=13 (maior que o tamanho do array, então paramos aqui).

A sequência de incrementos que usaremos, em ordem decrescente, é 4,1 (obtidos de h = (h − 1) / 3: (13 − 1) / 3 = 4, (4 − 1) / 3 = 1).

Primeira Passada (h = 4):

- O item 'O' (posição 1) é comparado com 'N' (posição 5) e trocados.
- O item 'R' (posição 2) é comparado e trocado com 'A' (posição 6).
- Vetor após h = 4: N,A,D,E,O,R.

Segunda Passada (h = 1):

- Corresponde ao algoritmo de inserção direta. Apenas trocas locais serão executadas.
- Os elementos são rearrumados sequencialmente. Por exemplo, 'N', 'D' e 'O', nas posições 1, 3 e 5 são rearrumados para resultar em 'D', 'N' e 'O' nestas mesmas posições. Da mesma forma, 'A', 'E', 'R', nas posições 2, 4 e 6 são comparados e mantidos em seus lugares.
- Vetor após h = 1: A,D,E,N,O,R.

Vetor Ordenado: A,D,E,N,O,R.

Análise de Complexidade

A razão da eficiência do Shellsort ainda não é totalmente conhecida, e ninguém foi capaz de analisar o algoritmo completamente devido a problemas matemáticos complexos relacionados à sequência de incrementos. No entanto, sabe-se que cada incremento não deve ser múltiplo do anterior.

Existem algumas conjecturas sobre o número de comparações para a sequência de Knuth:

- Conjectura 1: $C(n) = O(n\ 1.25)$
- Conjectura 2: $C(n) = O(n\ (\ln n)^2)$

Vantagens e Desvantagens

Vantagens:

- É uma ótima opção para arquivos de tamanho moderado (da ordem de 5000 registros).
- Sua implementação é simples e requer uma quantidade pequena de código.
- O tempo de execução do algoritmo é sensível à ordem inicial do arquivo, o que garante um bom uso em sequências já ordenadas (herdado do método de inserção).

Desvantagens:

- O tempo de execução do algoritmo é sensível à ordem inicial do arquivo.
- O método não é estável, pois ele pode realizar troca de elementos de igual valor.

Quick Sort

O Quicksort é um algoritmo de ordenação interna considerado o mais rápido para uma ampla variedade de situações e é provavelmente o mais utilizado. Foi proposto por Hoare em 1960 e publicado em 1962.

Ideia Básica

A ideia fundamental do Quicksort é dividir o problema de ordenar um conjunto de 'n' itens em dois problemas menores. Esses problemas menores são então ordenados de forma independente, e os resultados são combinados para formar a solução final.

Princípio de Divisão e Conquista

O Quicksort se baseia no padrão de projeto "Divisão e Conquista" (Divide-and-Conquer). Este padrão é composto por três fases gerais:

- **Divisão:** Os dados de entrada são divididos em dois ou mais conjuntos separados (disjuntos).
- **Recursão:** Os problemas dos subconjuntos são resolvidos recursivamente.
- **Conquista:** As soluções dos subproblemas são obtidas e combinadas em uma única solução.

Processo de Partição

A parte mais delicada do Quicksort é o processo de partição. Um vetor v[esq..dir] é rearranjado escolhendo-se arbitrariamente um pivô x. O vetor v é dividido em duas partes:

- **Parte esquerda:** Contém chaves menores ou iguais a x.
- **Parte direita:** Contém chaves maiores ou iguais a x.

Algoritmo para o particionamento:

1. Escolha arbitrariamente um pivô x.
2. Percorra o vetor a partir da esquerda até que $v[i] \geq x$.
3. Percorra o vetor a partir da direita até que $v[j] \leq x$.
4. Troque v[i] com v[j].
5. Continue este processo até que os ponteiros i e j se cruzem.

Após a partição, os itens em v[esq],v[esq+1],...,v[j] são menores ou iguais ao pivô x, e os itens em v[i],v[i+1],...,v[dir] são maiores ou iguais ao pivô x.

Exemplo de Ordenação (Passo a Passo)

Vamos usar o exemplo fornecido no material com o vetor inicial: 3,6,4,5,1,7,2.

O pivô x é escolhido como o elemento central: v[(i + j) / 2].

Vetor Inicial: 3,6,4,5,1,7,2

Primeira partição:

- Pivô: 5 (elemento central)
- Após a partição, o vetor fica: 3,2,4,1,5,7,6
    - Elementos menores/iguais a 5: 3,2,4,1
    - Elementos maiores/iguais a 5: 7,6

Segunda partição:

- Aplicada aos subvetores. Considerando o subvetor 3,2,4,1:
    - Pivô: 2 (central)
    - Após a partição, o subvetor fica: 1,2,4,3
    - (O pivô 2 já está na posição correta)

Terceira partição:

- Continuando com o subvetor 4,3:
    - Pivô: 3
    - Após a partição, o subvetor fica: 3,4

Quarta partição:

- Agora o vetor está semi-ordenado. Considerando o subvetor à direita do 4: 5,7,6
- Pivô: 7
- Após a partição, o subvetor fica: 5,6,7.

Quinta partição:

- Considerando o subvetor 5,6:
- Após a partição, o subvetor fica: 5,6.

Vetor Final: 1,2,3,4,5,6,7

Análise de Complexidade

- **Melhor caso:** Ocorre quando cada partição divide o arquivo em duas partes iguais. A função de custo é $C(n) = 2C(n/2) + n = n \log n$. Isso acontece quando o pivô já está em sua posição correta.
- **Pior caso:** Ocorre quando, sistematicamente, o pivô é escolhido como um dos extremos de um arquivo já ordenado. Isso faz com que o procedimento seja chamado recursivamente n vezes, eliminando apenas um item a cada chamada. A função de custo é $C(n) = O(n^2)$. O pior caso pode ser evitado empregando pequenas modificações no algoritmo, como escolher a mediana de três itens quaisquer do vetor como pivô.
- **Caso médio:** De acordo com Sedgewick e Flajolet (1996, p. 17), o tempo de execução médio do Quicksort é $O(n \log n)$, com $C(n) \approx 1{,}386\, n \log n - 0{,}846n$.

Vantagens e Desvantagens

Vantagens:

- Extremamente eficiente para ordenar arquivos de dados.
- Requer apenas uma pequena pilha como memória auxiliar.
- Requer cerca de n log n comparações em média para ordenar n itens.

Desvantagens:

- Possui um pior caso de $O(n^2)$ comparações.
- Sua implementação é delicada e difícil; um pequeno erro pode levar a efeitos inesperados para algumas entradas de dados.
- O método não é estável (não preserva a ordem de registros de chaves iguais).

Resumão

Algoritmo	Notação Big O (Melhor/ Médio/ Pior Caso)	É Estável?	Vantagens	Desvantagens
Quicksort	O(n log n) / O(n log n) / O(n²)	Não	Extremamente eficiente para ordenar arquivos de dados. Requer uma pequena pilha como memória auxiliar. Requer cerca de n log n comparações em média.	Pior caso de O(n²) comparações. A implementação é delicada e difícil. Não é estável.
Shellsort	Conjecturas: O(n^1.25) e O(n (ln n)²)	Não	Ótima opção para arquivos de tamanho moderado. Implementação simples e requer pouco código. Bom para sequências já ordenadas.	O tempo de execução é sensível à ordem inicial do arquivo. Não é estável.
Insert Sort	O(n) / O(n²) / O(n²)	Sim	A ser utilizado quando o arquivo está "quase" ordenado. Bom método para adicionar poucos itens a um arquivo já ordenado.	São feitos muitos deslocamentos cada vez que um elemento é colocado na sua posição correta.
Select Sort	O(n²) para todos os casos	Não	Adequado para arquivos com registros grandes. Custo linear no tamanho da entrada para o número de movimentos. Poucos movimentos de registros.	O número de movimentos não diminui se o arquivo já estiver ordenado. O algoritmo não é estável.
Bubblesort	O(0) / O(n²) / O(n²)	Sim	É o processo mais simples de entender e implementar. Adequado para arquivos pequenos.	Custo alto. O número de comparações não se altera se o vetor já estiver parcialmente ordenado.

Breve resumo do funcionamento de cada método:

- **Bubblesort:** É o método mais simples. Ele percorre o vetor várias vezes, comparando pares de elementos adjacentes. Se eles estiverem fora de ordem, são trocados de posição. Esse processo faz com que os elementos maiores "borbulhem" para o final do vetor a cada passagem.

- **Select Sort (Seleção Direta):** Este método varre o segmento não ordenado do vetor para encontrar o menor (ou maior) valor. Uma vez encontrado, esse elemento é trocado com o primeiro elemento do segmento. O processo é repetido, diminuindo o tamanho do segmento a cada passo, até que o vetor esteja completamente ordenado.

- **Insert Sort (Inserção Direta):** Funciona como a organização de um baralho de cartas. Cada elemento, a partir do segundo, é pego e inserido em sua posição correta na porção do vetor que já está ordenada. Para fazer a inserção, os elementos maiores são deslocados para a direita para abrir espaço.

- **Insert Sort (Inserção Direta):** Funciona como a organização de um baralho de cartas. Cada elemento, a partir do segundo, é pego e inserido em sua posição correta na porção do vetor que já está ordenada. Para fazer a inserção, os elementos maiores são deslocados para a direita para abrir espaço.

- **Shellsort:** É uma extensão do algoritmo de ordenação por inserção. Ele divide o vetor em vários subconjuntos com base em uma distância (ou incremento) `h` e aplica o método de inserção nesses subconjuntos. A distância `h` é progressivamente diminuída até que se torne 1, momento em que o algoritmo realiza uma ordenação por inserção final no vetor inteiro.

- **Quicksort:** Utiliza a estratégia de "dividir e conquistar". Ele escolhe um elemento pivô e rearranja o vetor de forma que todos os elementos menores ou iguais ao pivô fiquem de um lado, e os maiores ou iguais fiquem do outro. Esse processo de partição é aplicado recursivamente aos subvetores até que todo o conjunto esteja ordenado.